

数字电路设计 第 3 章

— 数字设计基础 (DDCA Ch1) 考试学习笔记

考试重要性：★★★ | 基础章节，为后续章节打基础 | 综合整理自 DDCA ARM Edition Ch1 pp.1–120（共 120 页幻灯片）

注意：异步时序逻辑不考，布尔方程自动化简不考

内容大纲（页码对应幻灯片页号）：

- 一、数字抽象与管理复杂度[pp.5–12]
- 二、数制系统：二进制、十进制与转换[pp.18–31]
- 三、十六进制与八进制[pp.32–40]
- 四、字节、半字节与二进制加法[pp.37–46]
- 五、有符号二进制数与补码运算★★★★ [pp.47–68]
- 六、逻辑门与真值表★★★★ [pp.69–79]
- 七、逻辑电平与噪声容限[pp.80–89]
- 八、CMOS 晶体管与逻辑门实现★★★★ [pp.90–113]
- 九、功耗分析★★★★ [pp.116–119]
- 附录 A：逻辑门真值表速查
- 附录 B：数制系统对照表
- 附录 C：全加器真值表与习题 [p.120]

一、数字抽象与管理复杂度 [pp.5–12]

抽象 (Abstraction) [p.5–6]

- **抽象** (Abstraction)：为某种目的，对一个概念或一种现象包含的信息进行过滤，移除不相关的信息，只保留与最终目的相关的信息。抓主要矛盾，抓矛盾的主要方面。 [p.6]
- 从底层到高层的抽象层级：electrons（电子）→ diodes（二极管）→ transistors（晶体管）→ amplifiers（放大器）→ NOT gates / AND gates（逻辑门）→ memories（存储器）→ adders（加法器）→ controllers（控制器）→ datapaths（数据通路）→ registers（寄存器）→ instructions（指令）→ device drivers（设备驱动）→ programs（程序） [p.6]

约束 (Discipline) [p.7]

- **约束** (Discipline)：有意识地限制设计选择。 [p.7]
- **数字约束** (Digital discipline)：使用离散电压代替连续电压，设计更简单、可构建更复杂系统。数字系统正在取代模拟前身：数码相机、数字电视、手机、CD 等。 [p.7]
- 1930 年，Vannevar Bush 造出世界上首台模拟电子计算机。 [p.7]

三 Y 原则 (The Three-Y's) [pp.8–9]

- **层次化** (Hierarchy)：系统划分为模块和子模块。 [p.9]
- **模块化** (Modularity)：每个模块有明确定义的功能和接口。 [p.9]
- **规整化** (Regularity)：鼓励一致性，使模块易于复用。 [p.9]

实例：燧发枪 (Flintlock Rifle) [pp.10–12]

- 层次化：三大模块 — lock（枪机）、stock（枪托）、barrel（枪管）；枪机子模块：trigger（扳机）、hammer（击锤）、flint（燧石）、frizzen（打火板）。 [p.10]
- 模块化：枪托功能是安装枪管和枪机，接口是安装销的长度和位置。 [p.12]
- 规整化：可互换零件。 [p.12]

二、数制系统：二进制、十进制与转换 [pp.13–31]**数字抽象与离散值 [pp.13–16]**

- 大多数物理变量是连续的量（导线上的电压、振荡频率、物体的位置）。数字抽象考虑离散的子集。 [p.13]
- **Charles Babbage** (1834–1871)：设计了分析机 (Analytical Engine)，被认为是第一台数字计算机，用机械齿轮表示离散值 (0–9)。Babbage 去世前未完成。 [p.14]
- **Ada Lovelace** (阿达)：1840 年编写了伯努利数算法，被认为是第一个程序员。 [p.15]
- **二进制值** (Binary Values)：两个离散值 — 1 (TRUE, HIGH) 和 0 (FALSE, LOW)。**比特** (Bit)：二进制数字。 [p.16]
- **George Boole** (1815–1864)：引入二进制变量和三种基本逻辑运算 AND、OR、NOT。著作《思维规律的研究》(1854)。 [p.17]

二进制与十进制转换 [pp.18–29]

- 二进制转十进制： $1101_2 = 1 \times 2^3 + 1 \times 2^2 + 0 \times 2^1 + 1 \times 2^0 = 13_{10}$ [p.19]
- 例： $10011_2 = 16 \times 1 + 8 \times 0 + 4 \times 0 + 2 \times 1 + 1 \times 1 = 19_{10}$ [p.23]

十进制转二进制：两种方法 [pp.24–29]

- 方法一：找最大的 2 的幂，减去并重复。例 53_{10} ：
 - $53 - 32 = 21$ (用 $32 = 2^5$, bit5=1); $21 - 16 = 5$ (bit4=1); $5 - 4 = 1$ (bit2=1); $1 - 1 = 0$ (bit0=1)
 - 结果： 110101_2 [p.26]
- 方法二：反复除以 2，余数依次填入更高位。例 53_{10} ：
 - $53/2 = 26$ R1; $26/2 = 13$ R0; $13/2 = 6$ R1; $6/2 = 3$ R0; $3/2 = 1$ R1; $1/2 = 0$ R1
 - 结果： 110101_2 (从最后一个余数开始读取) [p.26]
- 例： $75_{10} = 64 + 8 + 2 + 1 = 1001011_2$ [p.28–29]

2 的幂次速记 [p.20–21]

- $2^0 = 1, 2^1 = 2, 2^2 = 4, 2^3 = 8, 2^4 = 16, 2^5 = 32, 2^6 = 64, 2^7 = 128$
- $2^8 = 256, 2^9 = 512, 2^{10} = 1024, 2^{11} = 2048, 2^{12} = 4096, 2^{13} = 8192, 2^{14} = 16384, 2^{15} = 32768$
- **建议背诵到 2^9** 。 [p.21]

二进制范围 [pp.30–31]

- N 位十进制数： 10^N 种可能值，范围 $[0, 10^N - 1]$ [p.31]
- **N 位二进制数： 2^N 种可能值，范围 $[0, 2^N - 1]$** [p.31]
- 例：3 位二进制 $2^3 = 8$ 种可能值，范围 $[0, 7] = [000_2, 111_2]$ [p.31]

三、十六进制与八进制 [pp.32–40]

十六进制 (Hexadecimal) [pp.32–36]

- **十六进制** (Hexadecimal, Hex): 基数为 16, 是二进制的简写形式。 [p.34]
- 数字 0–9 (值 0–9), 字母 A–F (值 10–15)。 [p.32]
- **Hex** → **二进制**: 每位 Hex → 4 位二进制。 [p.33]
 - 例: $4AF_{16}$ (即 $0x4AF$) → $0100\ 1010\ 1111_2$ [p.36]
- **Hex** → **十进制**: 按权展开。 [p.36]
 - 例: $4AF_{16} = 16^2 \times 4 + 16^1 \times 10 + 16^0 \times 15 = 1199_{10}$ [p.36]
- 前缀: 通常用 0x 前缀表示十六进制数, 如 $0x4AF$ 。 [p.35]

2 的大幂次估算 [pp.38–40]

- $2^{10} \approx 1$ kilo (千, 1024) [p.38]
- $2^{20} \approx 1$ mega (兆, 1,048,576) [p.38]
- $2^{30} \approx 1$ giga (吉, 1,073,741,824) [p.38]
- $2^{40} \approx 1$ tera (太, TB); $2^{50} \approx 1$ peta (拍, PB); $2^{60} \approx 1$ exa (艾, EB); $2^{70} \approx 1$ zetta (泽, ZB); $2^{80} \approx 1$ yotta (尧, YB) [p.38]
- 估算技巧: $2^{24} = 2^4 \times 2^{20} \approx 16$ million; 32 位变量可表示 $2^2 \times 2^{30} \approx 4$ billion 种值。 [p.40]

四、字节、半字节与二进制加法 [pp.37–46]

比特、字节与半字节 [p.37]

- **最高有效位** (Most Significant Bit, msb): 最左边的位。 [p.37]
- **最低有效位** (Least Significant Bit, lsb): 最右边的位。 [p.37]
- **字节** (Byte): 8 位, 如 10010110。 [p.37]
- **半字节** (Nibble): 4 位, 一个字节可分为两个 nibble。 [p.37]
- **最高有效字节** (Most Significant Byte) 和 **最低有效字节** (Least Significant Byte)。 [p.37]

二进制加法 [pp.41–45]

- 与十进制加法类似, 考虑进位 (carry)。 [p.42]
- 例: $1011_2 + 0011_2 = 1110_2$ (含进位) [p.42]
- 例: $1001_2 + 0101_2 = 1110_2$ [p.44]
- 例: $1011_2 + 0110_2 = 10001_2$ (产生溢出!) [p.45]

溢出 (Overflow) [p.46]

- **溢出** (Overflow): 当结果太大, 无法用可用位数表示时发生。 [p.46]
- 数字系统工作在固定位数上。上例 $11 + 6$ 在 4 位二进制中产生溢出。 [p.46]

五、有符号二进制数与补码运算 ★★★ [pp.47–68]

原码 (Sign/Magnitude) [pp.48–51]

- 1 位符号位 + N-1 位数值位。最高位 (msb) 为符号位: 0 = 正, 1 = 负。 [p.48]

- 例 (4 位): $+6 = 0110$, $-6 = 1110$ [p.49]
- N 位原码范围: $[-(2^{N-1} - 1), 2^{N-1} - 1]$ [p.49]
- 原码的问题: [p.51]
 - 加法不工作: $-6 + 6 = 1110 + 0110 = 10100$ (错误!)
 - 0 有两种表示: 1000 (-0) 和 0000 ($+0$)

补码 (Two's Complement) ★★★ [pp.52–60]

- 优点: 加法正确工作; 0 只有一种表示; 符号位参与运算。 [p.52]
- 定义: $[x]_{\text{补}} = M + x \pmod{M}$, 其中 $M = 2^N$ (模)。 [p.52]
- 例: 机器字长 4 位, $x = -3$, $[x]_{\text{补}} = 2^4 + (-3) \pmod{2^4} = (1101)_2$ [p.52]

补码的表示范围 [pp.53–54]

- msb 的权重为 -2^{N-1} (而不是 $+2^{N-1}$)。 [p.54]
- 4 位补码: 最大正数 $0111 = +7$, 最大负数 $1000 = -8$ [p.54]
- msb 仍然指示符号 ($1 = \text{负数}$, $0 = \text{正数}$)。 [p.54]
- **N 位补码范围:** $[-2^{N-1}, 2^{N-1} - 1]$ 。注意: 比原码多一个负数 (-0 变为 -8 (对 4 位))。 [p.54]

求补操作 (Taking the Two's Complement) [pp.55–58]

- **求补操作** 翻转补码数的符号。 [p.55]
- 方法: [p.56]
 1. 所有位取反 (invert the bits)
 2. 加 1 (add 1)
- 例: $3_{10} = 0011_2$, 翻转符号:
 - 取反: 1100
 - 加 1: $1101 = -3_{10}$ [p.56]
- 例: $6_{10} = 0110_2$:
 - 取反: 1001 ; 加 1: $1010_2 = -6_{10}$ [p.58]
- 例: 已知补码 1001_2 , 求十进制值:
 - 取反: 0110 ; 加 1: $0111_2 = 7_{10}$, 所以原值为 -7_{10} [p.58]

补码加法 [pp.59–60]

- 符号位参与运算, 进位超出位宽的位忽略。 [p.59]
- 例: $6 + (-6)$: $0110 + 1010 = 1\ 0000$ (忽略进位, 结果为 0) [p.60]
- 例: $-2 + 3$: $1110 + 0011 = 1\ 0001 = 1$ (忽略进位) [p.60]

符号扩展与零扩展 [pp.61–63]

- **符号扩展** (Sign-Extension): 将符号位复制到新增的高位, 数值不变。 [p.62]
 - 例: 4 位 $3 = 0011 \rightarrow$ 8 位 00000011 [p.62]
 - 例: 4 位 $-5 = 1011 \rightarrow$ 8 位 11111011 [p.62]
- **零扩展** (Zero-Extension): 高位补零, 负数会改变数值。 [p.63]
 - 例: 4 位 $1011_2 = -5_{10}$, 零扩展为 8 位 $00001011_2 = 11_{10}$ (值变了!) [p.63]

数制系统对照 [pp.64–68]

- 三种系统的 4 位表示对比: [p.64–65]

数制系统	范围	4 位示例	特点
Unsigned (无符号)	$[0, 2^N - 1]$	0111 = 7, 1000 = 8	只有非负数
Sign/Magnitude (原码)	$[-(2^{N-1}-1), 2^{N-1}-1]$	0111 = 7, 1111 = -7	有 ± 0 ; 加法不工作
Two's Complement (补码)	$[-2^{N-1}, 2^{N-1}-1]$	0111 = 7, 1001 = -7	0 唯一; 加法正确

- **移码** (Bias/Offset code): $[x]_{\text{移}} = 2^{n-1} + x$ (通常用于表示浮点数阶码), 本质是补码的符号位取反。 [p.66]
- **反码** (One's Complement): 正数不变, 负数 = 原码数值位取反; 通常作为原码与补码之间的过渡码。 [p.66]

六、逻辑门与真值表 ★★★ [pp.69–79]

逻辑门概述 [p.69]

- 执行逻辑函数: NOT (非)、AND (与)、OR (或)、NAND (与非)、NOR (或非) 等。 [p.69]
- 单输入门: NOT gate, Buffer (缓冲器) [p.69]
- 双输入门: AND, OR, XOR, NAND, NOR, XNOR [p.69]
- 多输入门: 可扩展到 3 个及以上输入端。 [p.69]
- 多输入 XOR: 奇校验 (Odd parity)。 [p.77]

单输入逻辑门 [pp.70–71]

- **NOT (非门)**: $Y = \bar{A}$ 。真值表: $A = 0 \Rightarrow Y = 1$; $A = 1 \Rightarrow Y = 0$ [p.71]
- **BUF (缓冲器)**: $Y = A$ 。真值表: $A = 0 \Rightarrow Y = 0$; $A = 1 \Rightarrow Y = 1$ [p.71]

双输入基本逻辑门 [pp.72–75]

- **AND (与门)**: $Y = AB$ 。只有全 1 才输出 1。 [p.73]
- **OR (或门)**: $Y = A + B$ 。只要有 1 就输出 1。 [p.73]
- **XOR (异或门)**: $Y = A \oplus B = A\bar{B} + \bar{A}B$ 。输入不同时输出 1。 [p.75]
- **NAND (与非门)**: $Y = \overline{AB}$ 。AND 取反。 [p.75]
- **NOR (或非门)**: $Y = \overline{A + B}$ 。OR 取反。 [p.75]
- **XNOR (同或门)**: $Y = \overline{A \oplus B}$ 。输入相同时输出 1。 [p.75]

A	B	AND	OR	XOR	NAND	NOR	XNOR
0	0	0	0	0	1	1	1
0	1	0	1	1	1	0	0
1	0	0	1	1	1	0	0
1	1	1	1	0	0	0	1

表 1: 双输入逻辑门真值表总览

多输入逻辑门 [pp.76–77]

- **NOR3**: $Y = \overline{A + B + C}$ 。全 0 时输出 1, 其余为 0。 [p.77]
- **AND3**: $Y = ABC$ 。全 1 时输出 1, 其余为 0。 [p.77]

七、逻辑电平与噪声容限 [pp.80–89]

逻辑电平 [pp.80–81]

- 离散电压表示 1 和 0: $0 = \text{GND} (0\text{V})$, $1 = V_{DD}$ (如 5V)。 [p.80]
- 为 1 和 0 设定了电压范围, 且输入和输出电压范围不同, 以容忍噪声。 [p.81]

噪声与静态约束 [pp.82–85]

- **噪声** (Noise): 任何降低信号质量的因素 — 电阻、电源噪声、相邻导线耦合等。 [p.82]
- **静态约束** (Static Discipline): 输入逻辑有效的前提下, 每个电路元件必须产生逻辑有效的输出。 [p.83]
- 输出特性: V_{OH} (输出高电平最小值)、 V_{OL} (输出低电平最大值) [p.84–85]
- 输入特性: V_{IH} (输入高电平最小值)、 V_{IL} (输入低电平最大值) [p.84–85]
- **禁止区** (Forbidden Zone): V_{IL} 与 V_{IH} 之间的电压区域。 [p.84]
- **高噪声容限**: $NM_H = V_{OH} - V_{IH}$ [p.85]
- **低噪声容限**: $NM_L = V_{IL} - V_{OL}$ [p.85]
- 噪声容限越大, 允许的噪声越大, 电路的抗干扰能力越好。 NM_H 和 NM_L 必须大于 0。 [p.85]

DC 传输特性 [pp.86–87]

- **理想缓冲器**: $V(Y)$ 在 $V_{DD}/2$ 处跳变, $NM_H = NM_L = V_{DD}/2$ [p.86]
- **实际缓冲器**: 有渐变区, 单位增益点 (Unity Gain Points) ($\text{Slope} = 1$) 定义了 V_{IL} 和 V_{IH} 。 $NM_H, NM_L < V_{DD}/2$ [p.86–87]

V_{DD} 降压趋势 [pp.88–89]

- 1970–80 年代: $V_{DD} = 5\text{V}$ [p.88]
- 随后逐步下降: $3.3\text{V} \rightarrow 2.5\text{V} \rightarrow 1.8\text{V} \rightarrow 1.5\text{V} \rightarrow 1.2\text{V} \rightarrow 1.0\text{V} \dots\dots$ [p.88]
- 原因: 避免烧毁微型晶体管 + 节省功耗。 [p.88]
- 连接不同供电电压的芯片时需要小心。 [p.88]

八、CMOS 晶体管与逻辑门实现 ★★★ [pp.90–113]

晶体管基础 [pp.90–93]

- 逻辑门由晶体管构建。晶体管是三端口电压控制开关: d (漏极) 和 s (源极) 在 g (栅极) = 1 时导通 (ON), g=0 时关断 (OFF)。 [p.91]
- **Robert Noyce** (1927–1990): 集成电路的共同发明人, 共同创立了 Fairchild Semiconductor (1957) 和 Intel (1968)。 [p.92]
- **硅** (Silicon): 半导体。纯硅导电性差 (无自由电荷), 掺杂硅导电性好。 [p.93]
 - **n-type**: 自由负电荷 (电子, electrons) [p.93]
 - **p-type**: 自由正电荷 (空穴, holes) [p.93]

MOS 晶体管结构 [pp.94–97]

- **MOS** (金属氧化物硅): PolySilicon 栅极 + SiO_2 绝缘层 + 掺杂硅。 [p.94]
- **nMOS**: [p.95]
 - Gate=0: OFF (源漏之间无连接)

- Gate=1: ON (源漏之间形成 n 型导电沟道)
- **pMOS**: 与 nMOS 相反 [p.96]
 - Gate=0: ON
 - Gate=1: OFF
- pMOS 是 n 型衬底、p 沟道, 靠空穴的流动运送电流。 [p.96]

晶体管功能总结 [pp.97–98]

- **nMOS**: 导通时能很好地传递 0, 所以源极接 GND (下拉网络, pull-down network)。 [p.98]
- **pMOS**: 导通时能很好地传递 1, 所以源极接 V_{DD} (上拉网络, pull-up network)。 [p.98]

CMOS NOT 门 (反相器) [pp.99–100]

- 由一个 pMOS (P1) 和一个 nMOS (N1) 组成。 [p.100]
 - $A = 0$: P1=ON (上拉输出到 V_{DD}), N1=OFF, $Y = 1$ [p.100]
 - $A = 1$: P1=OFF, N1=ON (下拉输出到 GND), $Y = 0$ [p.100]
- CMOS 门的核心优势: 任何状态下上拉网络和下拉网络不同时导通, 静态功耗极低。 [p.100]

CMOS NAND 门 [pp.102–103]

- 两个并联 pMOS (P1, P2) 作为上拉网络 + 两个串联 nMOS (N1, N2) 作为下拉网络。 [p.103]
- $Y = \overline{AB}$ 。只有 $A = B = 1$ 时, N1 和 N2 同时导通下拉输出到 GND ($Y = 0$), 其它情况至少一个 pMOS 上拉到 V_{DD} ($Y = 1$)。 [p.103]

CMOS NOR 门 [pp.107–108]

- 两个串联 pMOS 作为上拉网络 + 两个并联 nMOS 作为下拉网络。 [p.108]
- $Y = \overline{A + B}$ 。 [p.107]

CMOS 门通用结构 [p.106]

- 上拉网络 (pMOS 构成) 将输出拉到 V_{DD} ; 下拉网络 (nMOS 构成) 将输出拉到 GND。 [p.106]
- 构建规则: pMOS 网络和 nMOS 网络互为对偶 (串联 $\leftrightarrow$ 并联)。 [p.106]

传输门与伪 nMOS [pp.111–113]

- **传输门** (Transmission Gate): 组合 nMOS 和 pMOS, nMOS 传递 0 弱、pMOS 传递 1 弱, 组合后 0 和 1 都能很好地传递, 是更好的开关。 [p.111]
 - EN=1: 开关 ON, A 连接到 B
 - EN=0: 开关 OFF, A 与 B 断开
- **伪 nMOS 门** (Pseudo-nMOS): 用始终导通的弱 pMOS 取代上拉网络, pMOS 仅在下拉网络不拉低时才将输出拉高。晶体管数少, 但存在静态功耗。 [p.112–113]

摩尔定律 (Moore's Law) [pp.114–115]

- **Gordon Moore**: 1965 年观察到计算机芯片上的晶体管数量每年翻一番, 1975 年起修正为每两年翻一番。 [p.114]
- ”如果汽车按照计算机的发展周期发展, 劳斯莱斯今天将花费 100 美元, 每加仑跑一百万英里, 并且每年爆炸一次。” — Robert Cringely [p.115]

九、功耗分析 ★★★ [pp.116–119]

功耗概述 [p.116]

- 功耗 (Power) = 单位时间消耗的能量。 [p.116]
- 分为两类：**动态功耗** (Dynamic Power) 和 **静态功耗** (Static Power)。 [p.116]

动态功耗 [p.117]

- 产生原因：对晶体管栅极电容 C 充电所消耗的能量。 [p.117]
- 将电容 C 充电至 V_{DD} 需要能量 CV_{DD}^2 。 [p.117]
- 电路以频率 f 运行，电容每秒充放电 $f/2$ 次（从 1 放电到 0 不消耗能量）。 [p.117]
- 动态功耗公式： [p.117]

$$P_{\text{dynamic}} = \frac{1}{2}CV_{DD}^2f$$

- 关键参数： C = 电容， V_{DD} = 电源电压， f = 开关频率。 [p.117]
- 考试重点：功耗与 V_{DD}^2 成正比，降低 V_{DD} 可以大幅降低功耗。 [p.117]

静态功耗 [p.118]

- 产生原因：门电路不切换时也有静电流 I_{DD} （也称漏电流，leakage current）。 [p.118]
- 静态功耗公式： [p.118]

$$P_{\text{static}} = I_{DD} \cdot V_{DD}$$

- I_{DD} = 静态供电电流（漏电流）。 [p.118]

总功耗与计算示例 [p.119]

- 总功耗： $P_{\text{total}} = P_{\text{dynamic}} + P_{\text{static}} = \frac{1}{2}CV_{DD}^2f + I_{DD}V_{DD}$ [p.119]
- 例：无线手持计算机功耗估算 [p.119]
 - $V_{DD} = 1.2V$, $C = 20nF$, $f = 1GHz$, $I_{DD} = 20mA$
 - $P = \frac{1}{2}(20nF)(1.2V)^2(1GHz) + (20mA)(1.2V)$
 - $P = 14.4W + 0.024W \approx 14.4W$
 - 结论：在该频段，动态功耗占绝对主导，静态功耗可忽略。 [p.119]

附录 A：逻辑门真值表速查

A	B	NOT(A)	AND	OR	XOR	NAND	NOR	XNOR
0	0	1	0	0	0	1	1	1
0	1	1	0	1	1	1	0	0
1	0	0	0	1	1	1	0	0
1	1	0	1	1	0	0	0	1

表 2: 双输入逻辑门完整真值表

附录 B：数制系统对照表

十进制	二进制 (4-bit)	十六进制	无符号	补码 (4-bit)
0	0000	0x0	0	0
1	0001	0x1	1	1
2	0010	0x2	2	2
3	0011	0x3	3	3
4	0100	0x4	4	4
5	0101	0x5	5	5
6	0110	0x6	6	6
7	0111	0x7	7	7
8	1000	0x8	8	-8
9	1001	0x9	9	-7
10	1010	0xA	10	-6
11	1011	0xB	11	-5
12	1100	0xC	12	-4
13	1101	0xD	13	-3
14	1110	0xE	14	-2
15	1111	0xF	15	-1

表 3: 4-bit 二进制与各种数制对照

附录 C：全加器真值表与习题 [p.120]

- 习题：依据三 Y 原则，采用 nMOS 和 pMOS 设计一个带进位的一位加法器（全加器）。输入为 A、B 和进位 C_{in} ，输出为 S（和）和进位 C_{out} 。 [p.120]

A	B	C_{in}	S (Sum)	C_{out} (Carry)
0	0	0	0	0
0	0	1	1	0
0	1	0	1	0
0	1	1	0	1
1	0	0	1	0
1	0	1	0	1
1	1	0	0	1
1	1	1	1	1

表 4: 全加器 (Full Adder) 真值表

- 逻辑表达式： [p.120]

$$S = A \oplus B \oplus C_{in}$$
$$C_{out} = (A \cdot B) + (A \cdot C_{in}) + (B \cdot C_{in})$$
$$A \oplus B = (A \cdot \overline{B}) + (\overline{A} \cdot B)$$

- 提示：用 Digital 工具进行仿真验证。 [p.120]